# Clase 12: Internet de las Cosas Médicas (IoMT) y Protocolo MQTT

**Semana 12 | Corte 3 — Sensores, Telemetría Biomédica y Cadena de Frío Hospitalaria**

Prof. Jefferson Sarmiento Rojas

2026-10-14

## Table of contents


Objetivos de Aprendizaje . . . 2
1. Evolución Histórica: Del IoT al IoMT . . . 3
¿Qué diferencia al IoT tradicional del IoMT? . . . 3
2. Anatomía de un Dispositivo IoMT . . . 3
3. El Protocolo MQTT: Fundamentos y Comparación con HTTP . . . 4
¿Por qué no usar simplemente peticiones HTTP (GET / POST)? . . . 4
El Desacoplamiento Tridimensional de MQTT . . . 5
Niveles de Calidad de Servicio (QoS - Quality of Service) . . . 5
4. Sintaxis y Jerarquía de Tópicos MQTT . . . 6
Reglas de diseño de tópicos: . . . 6
5. Caso Clínico de Estudio: Cadena de Frío en Farmacia Hospitalaria . . . 6
Justificación Biomédica y Farmacéutica . . . 6
6. Arquitectura del Sistema . . . 7
Descripción de los Componentes: . . . 7
7. Práctica de Laboratorio: Implementación en Wokwi y HiveMQ . . . 8
Configuración de Bibliotecas en Wokwi (`libraries.txt`) . . . 8
Esquema de Hardware en Wokwi (`diagram.json`) . . . 8
Código Fuente en C++ (`sketch.ino`) . . . 10
8. Validación en Vivo con HiveMQ WebSocket Client . . . 16
9. Simulador WebSockets Incrustado en la Guía . . . 17
10. Reto Evaluativo y Entregables . . . 17
11. Referencias de Consulta . . . 18

**Resumen Ejecutivo de la Sesión**

En esta sesión inauguramos el **Tercer Corte (Datos, Inteligencia Artificial y Ética)** abordando el origen de la telemetría clínica: el **Internet de las Cosas Médicas (IoMT)**. Conoceremos la evolución del IoT desde sus inicios, la anatomía de un sistema embebido hospitalario y analizaremos a fondo el protocolo **MQTT** (Message Queuing Telemetry Transport), estándar de facto para la transmisión ligera y asíncrona de señales biomédicas. Finalmente, implementaremos una solución de ingeniería biomédica para el **monitoreo de cadena de frío en una nevera de farmacia hospitalaria**, asegurando la viabilidad térmica de vacunas y medicamentos termolábiles mediante microcontroladores (**XIAO ESP32-C3** en simulador Wokwi), sensores de temperatura y humedad (**DHT22**), actuadores (LED de alarma), y el broker en la nube **HiveMQ** junto a su cliente web interactivo.

---

## Objetivos de Aprendizaje

Al finalizar esta sesión, el estudiante estará en capacidad de:

1. **Reconocer los hitos históricos y conceptuales del IoT**, identificando su evolución hacia la **Salud 4.0 / 5.0** y el ecosistema **IoMT** (Internet of Medical Things).
2. **Distinguir la arquitectura de un nodo embebido en salud**: sensores, microcontroladores, enlaces de comunicación, actuadores y brokers en la nube.
3. **Comprender y contrastar el protocolo MQTT** frente a esquemas convencionales como HTTP/REST, dominando el paradigma *Publish/Subscribe*, el desacoplamiento tridimensional y los niveles de Calidad de Servicio (**QoS 0, 1 y 2**).
4. **Diseñar esquemas jerárquicos de tópicos MQTT** aplicados a la telemetría y control de dispositivos en instalaciones hospitalarias.
5. **Programar y simular un sensor de temperatura y humedad (DHT22) sobre un XIAO ESP32-C3 en Wokwi**, publicando telemetría en JSON y recibiendo órdenes remotas para conmutar actuadores.
6. **Verificar y auditar en tiempo real la publicación y suscripción de datos** utilizando el cliente web oficial HiveMQ WebSockets Client.

---

## 1. Evolución Histórica: Del IoT al IoMT

El Internet de las Cosas no nació de la noche a la mañana; es el resultado de décadas de convergencia entre redes de computadores, miniaturización electrónica y sensores de bajo costo.

| Año | Hito Tecnológico | Descripción y Relevancia |
|---|---|---|
| **1982** | **Máquina de Coca-Cola (Carnegie Mellon)** | Estudiantes de ciencias de la computación |
| **1990** | **Internet Toaster (John Romkey)** | Presentada en la conferencia INTEROP, fu |
| **1999** | **Kevin Ashton & MIT Auto-ID** | Ashton acuña formalmente la frase *"Interr* |
| **2008** | **Punto de Inflexión Poblacional** | Cisco Systems estima que el número de dis |
| **2016** | **Botnet Mirai** | Ataque masivo DDoS que infectó cientos d |
| **2020 - 2026+** | **IoMT, AIoT y TinyML** | Convergencia de biosensores portátiles (we |

### ¿Qué diferencia al IoT tradicional del IoMT?

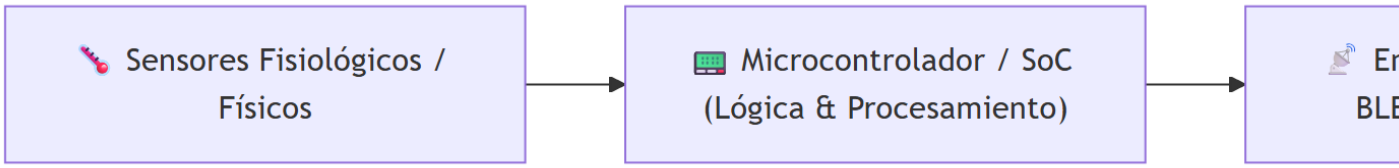
En una aplicación de IoT convencional (como un bombillo inteligente o un termostato doméstico), una falla temporal o una pérdida de paquetes resulta tolerable. En el **IoMT (Internet of Medical Things)**:

- **Criticidad de vida o muerte:** La pérdida del reporte de una arritmia cardíaca, la falla de un ventilador mecánico o una excursión térmica no detectada en vacunas puede costar vidas humanas o pérdidas millonarias.
- **Normativa estricta:** Dispositivos y software deben ceñirse a estándares de farmacovigilancia, FDA, Invima, normativas de dispositivos médicos (ISO 13485) y privacidad de datos (Ley de Habeas Data 1581 de 2012, HIPAA, GDPR).
- **Integridad de datos:** Se requiere trazabilidad inmutable y confirmación de entrega en la red.

---

## 2. Anatomía de un Dispositivo IoMT

En su nivel más elemental, cualquier dispositivo de Internet de las Cosas Médicas se compone de cuatro capas interconectadas:




1. **Sensores (Captura):** Transductores que transforman magnitudes físicas (temperatura, humedad, voltaje galvánico, fotopletismografía PPG, acelerometría) en señales eléctricas calibradas.
2. **Microcontrolador / SoC (Computación en el Borde):** Circuitos integrados de bajo consumo (como Seeed Studio XIAO ESP32-C3 o ESP32) que ejecutan el firmware, leen los sensores y ejecutan la lógica local de control (`if / else`).
3. **Conectividad (Transporte):** Módulos de radiofrecuencia (Wi-Fi IEEE 802.11, Bluetooth Low Energy BLE, LoRaWAN, Cellular NB-IoT) que transportan los paquetes.
4. **Broker / Plataforma en la Nube:** Servidor central encargado de recibir los mensajes de múltiples dispositivos, enrutarlos, almacenarlos y accionar alarmas en tiempo real.
5. **Actuadores & Alarmas:** Dispositivos de salida (LEDs de alerta, relés para encendido de refrigeradores auxiliares, sirenas sonoras o indicadores visuales).

---

## 3. El Protocolo MQTT: Fundamentos y Comparación con HTTP

### ¿Por qué no usar simplemente peticiones HTTP (GET / POST)?

En aplicaciones web convencionales, el protocolo estándar es **HTTP/1.1** o **HTTP/2**, basado en un modelo **Cliente-Servidor de Petición/Respuesta (Request/Response)**. Aunque HTTP es excelente para páginas web y APIs REST, presenta graves deficiencias en microcontroladores y dispositivos médicos alimentados por batería:

| Característica | HTTP (REST) | MQT |
|---|---|---|
| **Paradigma** | Petición / Respuesta (Request / Response, Síncrono) | Publi |
| **Cabecera del paquete** | Pesada (mínimo centenas de bytes a varios KB con headers HTTP) | **Ultra** |
| **Consumo de Batería y CPU** | Alto (crea y destruye sockets TCP en cada reporte) | **Extre** |

| Característica | HTTP (REST) | MQT |
|---|---|---|
| **Modelo de Datos** | Sondeo constante (*Polling*) para enterarse de novedades | Empu |
| **Conocimiento mutuo** | El cliente debe conocer la dirección IP / URL exacta del servidor | **Desa** |

### El Desacoplamiento Tridimensional de MQTT

Una de las mayores virtudes pedagógicas y operativas de MQTT es que rompe la dependencia directa entre los dispositivos:

1. **Desacoplamiento Espacial:** El sensor que mide la temperatura no tiene idea de qué computadores, teléfonos o navegadores están leyendo su dato. Solo publica en un canal (*tópico*) en el Broker.
2. **Desacoplamiento Temporal:** El publicador y el suscriptor no tienen que estar conectados a la red al mismo tiempo. Mediante banderas como *Retained Messages* o sesiones persistentes, un suscriptor que se conecte después puede recibir de inmediato el último estado conocido.
3. **Desacoplamiento de Sincronización:** La operación de publicación es asíncrona y no bloqueante. El microcontrolador envía su lectura y continúa de inmediato ejecutando su ciclo de control sin esperar a que otros la procesen.

### Niveles de Calidad de Servicio (QoS - Quality of Service)

MQTT ofrece tres niveles nativos de garantía en la entrega de paquetes según la criticidad del dato:

- **QoS 0 — "At most once" (A lo sumo una vez / Fuego y olvido):**
  El mensaje se envía sin confirmación a nivel de aplicación. Se apoya únicamente en la confiabilidad de la capa TCP subyacente. Si se corta el enlace, el paquete se pierde. Es ideal para telemetría no crítica de alta frecuencia.
- **QoS 1 — "At least once" (Al menos una vez):**
  El emisor retransmite el mensaje hasta recibir un paquete de reconocimiento (`PUBACK`) del Broker. Se garantiza que el mensaje llega, pero si hay fluctuaciones de red pueden presentarse mensajes duplicados.
- **QoS 2 — "Exactly once" (Exactamente una vez):**
  Es el nivel más confiable y estricto. Se realiza un protocolo de apretón de manos (*handshake*) en 4 pasos (`PUBLISH` → `PUBREC` → `PUBREL` → `PUBCOMP`). Garantiza que el mensaje llega exactamente una sola vez, sin pérdidas ni duplicados. Es el nivel idóneo para eventos médicos críticos, tales como **dosificación de medicamentos por bombas de infusión** o **alarmas de parada cardiorrespiratoria**.

---

## 4. Sintaxis y Jerarquía de Tópicos MQTT

Los tópicos (*topics*) en MQTT son cadenas de texto organizadas en forma de árbol jerárquico separadas por diagonales (`/`). Son **sensibles a mayúsculas y minúsculas** (*case-sensitive*).

### Reglas de diseño de tópicos:

- No utilizar espacios ni caracteres especiales con tildes.
- Diseñar de lo más general a lo más específico (ej. `institución/servicio/equipo/variable`).
- **Wildcards (Comodines) para Suscripción:**
  - `+` (Comodín de un solo nivel): Reemplaza exactamente un nivel del árbol.
    - Ejemplo: `hospital/farmacia/+/temperatura` permite recibir la temperatura de todas las neveras de la farmacia (`nevera_01`, `nevera_02`, etc.).
  - `#` (Comodín multinivel): Debe ubicarse siempre al final del tópico y captura todos los subniveles inferiores.
    - Ejemplo: `hospital/farmacia/#` suscribe a todos los mensajes (temperatura, humedad, estados de puerta, alarmas y actuadores).

---

## 5. Caso Clínico de Estudio: Cadena de Frío en Farmacia Hospitalaria

### Justificación Biomédica y Farmacéutica

En los servicios de salud (farmacia central, oncología, vacunación, banco de sangre), la conservación de medicamentos biológicos, vacunas (antirrábica, influenza, COVID-19, hepatitis B) e insulinas exige una temperatura constante en el rango estricto de:

$$\mathbf{2.0}^{\circ}\text{C} \leq \text{Temperatura} \leq \mathbf{8.0}^{\circ}\text{C}$$

La humedad relativa debe controlarse típicamente entre el **30% y el 60%** para impedir la condensación y el deterioro de los empaques primarios.

- **Excursión térmica por calor ($> 8.0^{\circ}$C):** Ocasiona desnaturalización irreversible de proteínas en vacunas y pérdida total de efectividad terapéutica.
- **Excursión térmica por congelamiento ($< 2.0^{\circ}$C):** Las vacunas con adyuvantes a base de sales de aluminio se desestabilizan y pierden su inmunogenicidad.

---

## 6. Arquitectura del Sistema

La siguiente figura ilustra la arquitectura de monitoreo de la cadena de frío implementada para el hospital:

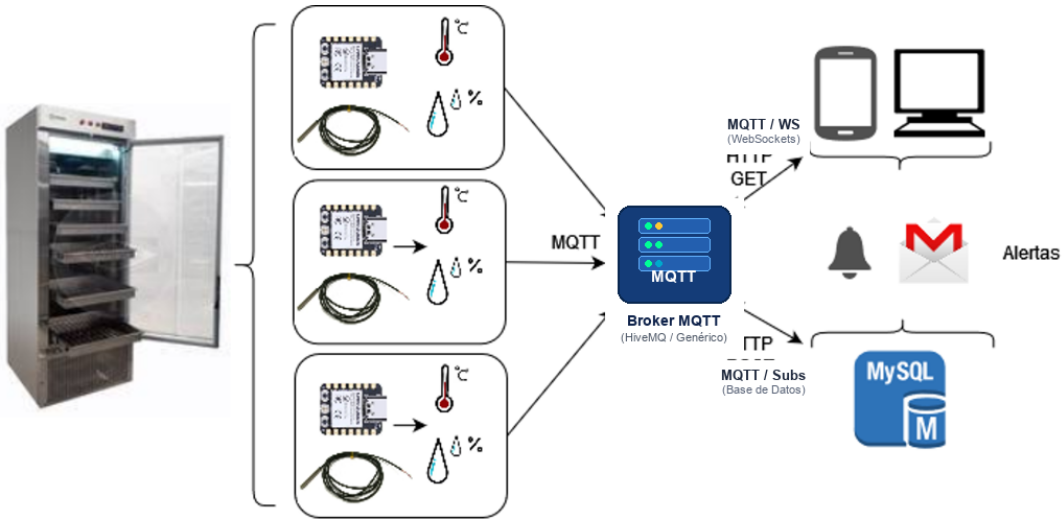



Figure 1: Arquitectura IoMT para Monitoreo de Cadena de Frío en Farmacia Hospitalaria

### Descripción de los Componentes:

1. **Refrigerador de Farmacia Hospitalaria:** Unidad médica con estantes para fármacos y vacunas.
2. **Nodos de Sensado (Microcontrolador ESP32 + Sensor DHT22 + Actuador LED):** Instrumentan la temperatura y humedad, transmiten telemetría y escuchan comandos de actuación.
3. **Flujo de Publicación (MQTT Publish):** El nodo se conecta a la red Wi-Fi y publica periódicamente sus lecturas en formato JSON hacia el broker.
4. **Broker MQTT Central (HiveMQ / Genérico):** Servidor central en la nube (`broker.hivemq.com`), encargado del enrutamiento asíncrono.
5. **Consumidores del Sistema:**
   - **Visualización en Tiempo Real:** Dashboard web oficial conectado mediante **MQTT sobre WebSockets** (HiveMQ WebSocket Client).
   - **Control Remoto Bidireccional:** El operador puede enviar órdenes de encendido o apagado (`on` / `off`) desde el navegador hacia el microcontrolador para activar alarmas o luces de inspección.
   - **Almacenamiento y Registro:** Servicios suscriptores en base de datos para trazabilidad y auditoría clínica.

## 7. Práctica de Laboratorio: Implementación en Wokwi y HiveMQ

En esta práctica programaremos el microcontrolador **Seeed Studio XIAO ESP32-C3** con un sensor **DHT22** y un **LED indicador** en el simulador en la nube **Wokwi**, transmitiendo datos en tiempo real al broker **HiveMQ** y probando la comunicación bidireccional desde el **HiveMQ WebSockets Client**.

> Plataforma de Simulación Virtual: Wokwi
>
> Realizaremos la simulación completa en Wokwi (ESP32 Simulator), utilizando la tarjeta **Seeed Studio XIAO ESP32-C3** con sensor **DHT22** (pin **D2**) y un **LED actuador** (pin **D4**). No se requiere hardware físico para realizar el laboratorio.

---

### Configuración de Bibliotecas en Wokwi (`libraries.txt`)

En la pestaña **Library Manager** de Wokwi, agregue las siguientes dos bibliotecas:

```
DHT sensor library
PubSubClient
```

---

### Esquema de Hardware en Wokwi (`diagram.json`)

En la pestaña `diagram.json` de Wokwi, pegue la siguiente configuración correspondiente al montaje en protoboard con **XIAO ESP32-C3**, sensor **DHT22** (conectado a **D2**), **LED** (conectado a **D4**) y resistencia de 1000 Ω:

```
{
  "version": 1,
  "author": "Uri Shaked",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-breadboard-half", "id": "bb1", "top": 25.8, "left": -122, "attrs": {} }
    {
```

```
      "type": "board-xiao-esp32-c3",
      "id": "esp",
      "top": 81.02,
      "left": -104.63,
      "rotate": 270,
      "attrs": {}
    },
    { "type": "wokwi-dht22", "id": "dht1", "top": 38.7, "left": 13.8, "attrs": {} },
    { "type": "wokwi-led", "id": "led1", "top": 44.4, "left": 119, "attrs": { "color": "red"
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 128.2,
      "left": 114.35,
      "rotate": 270,
      "attrs": { "value": "1000" }
    }
  ],
  "connections": [
    [ "bb1:3t.b", "bb1:tp.2", "red", [ "v0" ] ],
    [ "bb1:2t.b", "bb1:tn.1", "black", [ "v0" ] ],
    [ "bb1:14b.h", "bb1:11b.h", "red", [ "v0" ] ],
    [ "bb1:11b.g", "bb1:tp.9", "red", [ "v0" ] ],
    [ "bb1:16b.h", "bb1:17b.h", "black", [ "v0" ] ],
    [ "bb1:17b.i", "bb1:20b.i", "black", [ "v0" ] ],
    [ "bb1:20b.h", "bb1:tn.16", "black", [ "v0" ] ],
    [ "bb1:3b.i", "bb1:15b.i", "blue", [ "v0" ] ],
    [ "bb1:25t.c", "bb1:22t.c", "black", [ "v0" ] ],
    [ "bb1:tn.18", "bb1:22t.b", "black", [ "v0" ] ],
    [ "bb1:5b.j", "bb1:26b.j", "gold", [ "v0" ] ],
    [ "dht1:VCC", "bb1:14b.g", "", [ "$bb" ] ],
    [ "dht1:SDA", "bb1:15b.g", "", [ "$bb" ] ],
    [ "dht1:NC", "bb1:16b.g", "", [ "$bb" ] ],
    [ "dht1:GND", "bb1:17b.g", "", [ "$bb" ] ],
    [ "esp:D0", "bb1:1b.g", "", [ "$bb" ] ],
    [ "esp:D1", "bb1:2b.g", "", [ "$bb" ] ],
    [ "esp:D2", "bb1:3b.g", "", [ "$bb" ] ],
    [ "esp:D3", "bb1:4b.g", "", [ "$bb" ] ],
    [ "esp:D4", "bb1:5b.g", "", [ "$bb" ] ],
    [ "esp:D5", "bb1:6b.g", "", [ "$bb" ] ],
    [ "esp:D6", "bb1:7b.g", "", [ "$bb" ] ],
    [ "esp:D7", "bb1:7t.c", "", [ "$bb" ] ],
```

```
    [ "esp:D8", "bb1:6t.c", "", [ "$bb" ] ],
    [ "esp:D9", "bb1:5t.c", "", [ "$bb" ] ],
    [ "esp:D10", "bb1:4t.c", "", [ "$bb" ] ],
    [ "esp:3V3", "bb1:3t.c", "", [ "$bb" ] ],
    [ "esp:GND", "bb1:2t.c", "", [ "$bb" ] ],
    [ "esp:5V", "bb1:1t.c", "", [ "$bb" ] ],
    [ "led1:A", "bb1:26t.b", "", [ "$bb" ] ],
    [ "led1:C", "bb1:25t.b", "", [ "$bb" ] ],
    [ "r1:1", "bb1:26b.h", "", [ "$bb" ] ],
    [ "r1:2", "bb1:26t.d", "", [ "$bb" ] ]
  ],
  "dependencies": {}
}
```

---

### Código Fuente en C++ (`sketch.ino`)

Copie y pegue el siguiente código en la pestaña `sketch.ino` de Wokwi. Este código incluye: * Generación automática del ID único de cliente MQTT utilizando la dirección MAC física del ESP32. * Conexión a la red Wi-Fi virtual `Wokwi-GUEST`. * Conexión con el broker gratuito **HiveMQ** (`broker.hivemq.com` en puerto `1883`). * Publicación periódica del JSON con temperatura, humedad y estado clínico de la cadena de frío. * Función de escucha (*callback*) para encender o apagar el actuador LED desde el cliente web mediante el tópico `Led` o `hospital/farmacia/nevera_01/led`.

```
// ============================================================================
// SISTEMA IoMT DE TELEMETRÍA Y CONTROL DE CADENA DE FRÍO - FARMACIA HOSPITALARIA
// Asignatura: Informática Médica (IFMER 2026-II) - Universidad del Rosario
// Docente: Prof. Jefferson Sarmiento Rojas
// Placa: Seeed Studio XIAO ESP32-C3 | Sensor: DHT22 en D2 | LED en D4
// Broker: HiveMQ (broker.hivemq.com:1883)
// ============================================================================

// 1. Importar librerías
#include "WiFi.h"          // Conexión WiFi
#include "PubSubClient.h"  // Cliente MQTT
#include "DHT.h"           // Sensor DHT22

// Pin de salida que gobierna el LED actuador
const int lamp = D4;
```

```
// 2. Configuración de red inalámbrica
const char* ssid = "Wokwi-GUEST";           // Red virtual de Wokwi
const char* password = "";                  // Sin contraseña en simulador
const char* mqtt_server = "broker.hivemq.com"; // Broker MQTT público y gratuito HiveMQ

// 3. Cliente WiFi y cliente MQTT
WiFiClient espClient;
PubSubClient client(espClient);
unsigned long lastSend;
int status = WL_IDLE_STATUS;

// Nombre de cliente MQTT generado automáticamente
String clientName;

// 4. Configuración del sensor
#define DHTPIN D2      // Pin de datos del DHT22 conectado a D2
#define DHTTYPE DHT22  // Referencia del sensor DHT22

DHT dht(DHTPIN, DHTTYPE);

float h, t;            // Variables de humedad y temperatura

// 5. Mensaje que se publica en formato JSON
String payload;

// ================================================================================
// GENERAR NOMBRE DE CLIENTE MQTT A PARTIR DE LA MAC DEL ESP32
// ================================================================================
void generarNombreCliente() {
  uint64_t mac = ESP.getEfuseMac();
  char nombre[30];

  snprintf(
    nombre,
    sizeof(nombre),
    "ESP32_%012llX",
    mac
  );

  clientName = String(nombre);

  Serial.print("Nombre cliente MQTT generado: ");
```

```
  Serial.println(clientName);
}

// ============================================================================
// DECLARACIÓN DE CALLBACK PARA MENSAJES ENTRANTES (ÓRDENES REMOTAS)
// ============================================================================
void callback(char* topic, byte* message, unsigned int length) {
  Serial.print("Llegó un mensaje del tópico: ");
  Serial.print(topic);
  Serial.print(". Mensaje: ");

  String messageTemp = "";
  for (int i = 0; i < length; i++) {
    Serial.print((char)message[i]);
    messageTemp += (char)message[i];
  }
  Serial.println();

  // Control remoto del actuador LED
  if (String(topic) == "hospital/farmacia/nevera_01/led" || String(topic) == "Led") {
    if (messageTemp == "on" || messageTemp == "ON") {
      digitalWrite(lamp, HIGH);
      Serial.println(">>> ACTUADOR: LED encendido por orden remota");
    }
    else if (messageTemp == "off" || messageTemp == "OFF") {
      digitalWrite(lamp, LOW);
      Serial.println(">>> ACTUADOR: LED apagado por orden remota");
    }
  }
}

// ============================================================================
// RECONEXIÓN AL BROKER MQTT
// ============================================================================
void reconnect() {
  while (!client.connected()) {
    Serial.print("Intentando conectar al broker HiveMQ...");

    if (client.connect(clientName.c_str())) {
      Serial.println(" ¡Conectado exitosamente!");

      // Suscripción a los tópicos de control remoto para el actuador LED
```

```
      client.subscribe("hospital/farmacia/nevera_01/led");
      client.subscribe("Led");
      Serial.println("Suscrito a los tópicos de control: Led y hospital/farmacia/nevera_01/le

    } else {
      Serial.print(" Fallo de conexión, rc=");
      Serial.print(client.state());
      Serial.println(" Intentando nuevamente en 3 segundos...");
      delay(3000);
    }
  }
}

// ================================================================================
// CONEXIÓN A LA RED INALÁMBRICA WI-FI
// ================================================================================
void setup_wifi() {
  delay(10);
  Serial.println();
  Serial.print("Conectando a la red: ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(200);
    Serial.print(".");
  }

  Serial.println("");
  Serial.print("Conexión WiFi establecida. Dirección IP asignada: ");
  Serial.println(WiFi.localIP());
}

// ================================================================================
// CONFIGURACIÓN INICIAL (SETUP)
// ================================================================================
void setup() {
  Serial.begin(9600);

  // Generar ID único usando la MAC
  generarNombreCliente();
```

```
  // Conectar a Wi-Fi
  setup_wifi();

  // Configurar servidor MQTT y función de escucha
  client.setServer(mqtt_server, 1883);
  client.setCallback(callback);

  // Inicializar periféricos
  pinMode(lamp, OUTPUT);
  dht.begin();

  lastSend = 0;
  delay(2000);
}

// ========================================================================
// LECTURA DEL SENSOR DHT22
// ========================================================================
void GetSensor() {
  h = dht.readHumidity();
  t = dht.readTemperature();

  if (isnan(h) || isnan(t)) {
    Serial.println("[ERROR] Falla al leer el sensor DHT22!");
    return;
  }
}

// ========================================================================
// ARMADO DEL MENSAJE EN FORMATO JSON
// ========================================================================
void CrearJson() {
  String temperatura = String(t, 2);
  String humedad = String(h, 2);

  // Construcción del paquete JSON
  payload = "{";
  payload += "\"temp\":";
  payload += temperatura;
  payload += ",";
  payload += "\"hum\":";
  payload += humedad;
```

```
  // Lógica médica para cadena de frío en farmacia: rango 2.0 °C a 8.0 °C
  if (t > 8.0 || t < 2.0) {
    payload += ",\"alerta\":true";
    payload += ",\"estado\":\"EXCURSION_TERMICA\"";
  } else {
    payload += ",\"alerta\":false";
    payload += ",\"estado\":\"NORMAL\"";
  }

  payload += "}";
}

// ============================================================================
// PUBLICACIÓN DE LA TELEMETRÍA
// ============================================================================
void PublicarMensaje() {
  char attributes[150];
  payload.toCharArray(attributes, 150);

  // Publicación en el tópico hospitalario y en el tópico general
  client.publish("hospital/farmacia/nevera_01/telemetria", attributes);
  client.publish("TopicURSensorTempHum", attributes);

  Serial.print("[MQTT Enviado] -> ");
  Serial.println(attributes);
}

// ============================================================================
// CICLO PRINCIPAL (LOOP)
// ============================================================================
void loop() {
  if (!client.connected()) {
    reconnect();
  }

  // Medir y publicar cada 2 segundos
  if (millis() - lastSend > 2000) {
    GetSensor();
    CrearJson();
    PublicarMensaje();
    lastSend = millis();
  }
```

```
  client.loop(); // Mantiene activa la conexión MQTT y procesa callbacks
}
```

---

## 8. Validación en Vivo con HiveMQ WebSocket Client

Para auditar y probar la solución no se requiere programar herramientas externas; utilizaremos directamente la herramienta oficial en la nube:

**https://www.hivemq.com/demos/websocket-client/**

**!** Procedimiento de Prueba Paso a Paso

### 1. Conexión al Broker

- Abra la página del cliente: https://www.hivemq.com/demos/websocket-client/.
- Verifique que el **Host** sea `broker.hivemq.com` y el **Port** sea `8884` (puerto seguro WSS).
- Haga clic en el botón verde **"Connect"**. El indicador cambiará a **"Connected"**.

### 2. Suscribirse a la Telemetría de la Nevera

- En la sección **Subscriptions**, haga clic en **"Add New Topic Subscription"**.
- Escriba el tópico: `hospital/farmacia/nevera_01/telemetria` (o también `TopicURSensorTempHum`).
- Deje QoS en `0` o `1` y haga clic en **"Subscribe"**.
- Inicie la simulación en Wokwi. Cada 2 segundos observará los paquetes JSON llegando en vivo:

```
{"temp":4.50,"hum":50.00,"alerta":false,"estado":"NORMAL"}
```

### 3. Control Remoto del Actuador (Encender / Apagar el LED)

- En la sección **Publish** del cliente HiveMQ:
  - **Topic:** `hospital/farmacia/nevera_01/led` (o `Led`)
  - **Message:** `on`
  - Haga clic en **"Publish"**.
    **Resultado en Wokwi:** El LED rojo conectado al pin D4 se **encenderá** de inmediato y en el monitor serial aparecerá el mensaje: `>>> ACTUADOR: LED encendido por orden remota`.

– Ahora cambie el mensaje a `off` y haga clic en **"Publish"**.
**Resultado en Wokwi:** El LED rojo se **apagará**.

**4. Simulación de Falla de Cadena de Frío**

- En Wokwi, haga clic sobre el sensor **DHT22**.
- Mueva el deslizador de temperatura a 15°**C** (simulando apertura prolongada de puerta o daño en el compresor).
- Observe en el cliente HiveMQ cómo el campo cambia automáticamente a:

```
{"temp":15.00,"hum":50.00,"alerta":true,"estado":"EXCURSION_TERMICA"}
```

---

## 9. Simulador WebSockets Incrustado en la Guía

Para comodidad durante la clase, también puedes probar la recepción y emisión de mensajes **directamente en esta página web**:

---

## 10. Reto Evaluativo y Entregables

Cada equipo de trabajo deberá completar las siguientes actividades:

1. **Configuración de Simulación en Wokwi:**
   - Montar el circuito con **XIAO ESP32-C3**, **DHT22** y **LED** en protoboard utilizando el `diagram.json` proporcionado.
   - Compilar y cargar el código C++ en `sketch.ino`.
2. **Validación con HiveMQ WebSocket Client:**
   - Conectarse a `broker.hivemq.com` desde la herramienta web.
   - Suscribirse al tópico de telemetría y documentar la llegada periódica de lecturas en formato JSON.
   - Encender y apagar el LED desde el navegador enviando las órdenes `on` y `off`.
3. **Simulación de Alarma Médica:**
   - Modificar la temperatura en Wokwi a un valor fuera de rango ($> 8.0$°C o $< 2.0$°C).

- Capturar la evidencia donde el JSON refleja `"alerta": true` y `"estado": "EXCURSION_TERMICA"`.

4. **Entregables:**
   - Enlace público de la simulación funcional en Wokwi.
   - Capturas de pantalla de la recepción de datos y control del LED desde el cliente web de HiveMQ.
   - Breve análisis de farmacovigilancia sobre el impacto de la pérdida de cadena de frío en medicamentos biológicos.

---

## 11. Referencias de Consulta


1. **HiveMQ Public MQTT Broker & WebSocket Client:** https://www.hivemq.com/demos/websocket-client/
2. **Wokwi Online Simulator for ESP32 and Arduino:** https://wokwi.com/
3. **OASIS Standard:** *MQTT Version 5.0 and 3.1.1 Specification.* https://mqtt.org/
4. **Organización Panamericana de la Salud (OPS/OMS):** *Cadena de frío de las vacunas: Manual del operador.* Módulo de Trazabilidad y Monitoreo de Temperatura.